THE REMOTE SUPPORT PROGRAM · DEPLOYMENT RUNBOOK

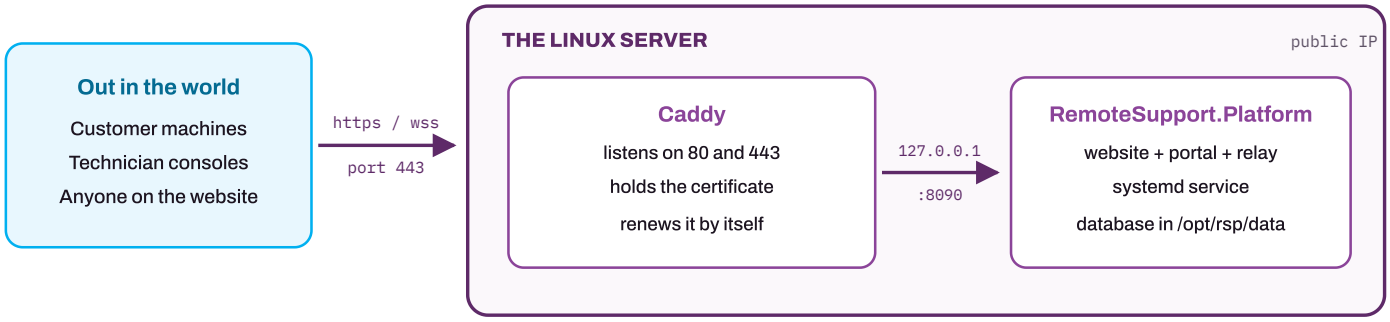
Putting the platform live on theremotesupportprogram.com

Everything needed to take the product from a LAN build to a public, TLS-secured server that customers anywhere can connect to. Written for a technician who has not worked on this codebase. Allow about an hour, most of it waiting for DNS.

Version 1 · 14 September 2026 Owner: Clint, Anywhere IT Source: docs/GO-LIVE.md

What you are building

The whole product — the public website, the customer portal and the relay that carries remote-control sessions — is **one program**, `RemoteSupport.Platform`. You are installing that one program on a Linux server, with a web server called Caddy in front of it holding the TLS certificate.



The platform listens only on the server's own loopback address, so nothing on the internet can reach it except through Caddy. That is deliberate: it means there is no way to reach the portal over unencrypted HTTP, because nothing is listening for it.

> **SHARED HOSTING WILL NOT WORK**
>
> This is not a PHP site. It is a long-running program that must listen on a port and hold thousands of WebSocket connections open. It needs a virtual machine with root access — xneelo Cloud, an xneelo self-managed server, or any Ubuntu VPS. A cPanel or shared-hosting account cannot run it at all.

Before you start

YOU NEED	NOTES
An account that can create a server	xneelo Cloud, or whichever provider Clint has chosen.
Access to the domain's DNS	`theremotesupportprogram.com` is registered at **Afrihost**, but its nameservers point at **Cloudflare** — the records are edited in the **Cloudflare dashboard**, not in Afrihost ClientZone.
A Windows PC with the repository	The folder `Remote Support`, plus the **.NET 8 SDK** at `C:\Program Files\dotnet`. The build must run on Windows — parts of the suite are WinForms. If Clint hands you a ready-made `dist\platform-linux` folder instead, add `-SkipBuild` to the command in step 3 and no SDK is needed.
An SSH key pair	If you do not have one, run `ssh-keygen -t ed25519` in PowerShell and use the contents of `C:\Users\<you>\.ssh\id_ed25519.pub`.

> **THE SHORT VERSION**
>
> Create an Ubuntu server → point two DNS records at it → run one command from the Windows PC → create the first tenant → hand the owner key back to Clint. Steps 3 to 5 are each a single command.

1 Create the server `AT THE HOSTING PROVIDER`

Create a virtual machine:

- **Ubuntu 24.04 LTS** (22.04 also works).
- **2 vCPU, 4 GB RAM, 40 GB disk** to start. The installed size is about 350 MB.
- Add **your SSH public key** during creation, so you can log in as `root` without a password.
- Leave the provider's firewall open on **22, 80 and 443**. The script closes everything else itself.

Write down the **public IP address**. Everything below refers to it as `<THE-IP>`.

> **SIZING**
>
> CPU is not what runs out first — **bandwidth is**. Every frame of every remote session passes through this machine, roughly 0.5 to 2 Mbit/s per active session. Check the traffic allowance on the plan before the number of technicians grows.

Check you can log in before going any further:

```
ssh root@<THE-IP>
```

2 Point the domain at it IN THE CLOUDFLARE DASHBOARD

Open the domain in Cloudflare and go to **DNS → Records**. Change exactly two records. Both already exist and point at the shared hosting that serves the holding page:

TYPE	NAME	VALUE	PROXY STATUS
A	`@`	<THE-IP>	**DNS only** (grey cloud)
A	`www`	<THE-IP>	**DNS only** (grey cloud)

DO NOT TOUCH THESE

- **The mail records.** `mail`, `smtp` and `webmail`, and every MX and TXT record, carry the company's email from a different server on purpose. Nothing about this move should change them.
- **The nameservers.** There is no need to move DNS hosting anywhere. Only the two A records above change.
- **The grey cloud.** Leave both records on *DNS only*. Switching on Cloudflare's orange cloud stops the certificate being issued and breaks the remote-support connection itself.

Changes usually appear within minutes but can take up to an hour. Confirm from the Windows PC before moving on — the certificate in step 3 cannot be issued until this answers with the server's IP:

```
nslookup theremotesupportprogram.com
```

3 Deploy ON THE WINDOWS PC

Open PowerShell in the repository folder (the one containing `build.ps1`) and run:

```
powershell -ExecutionPolicy Bypass -File deploy\Publish-Platform.ps1 -Server <THE-IP>
```

That one command builds the Linux payload, packs it, uploads it over SSH, and runs the installer on the server. The installer:

1. installs **Caddy** (the TLS front door) and **Google Chrome** (used server-side to turn health-check reports into PDFs);
2. installs the platform to `/opt/rsp` and runs it as the systemd service `remotesupport-platform`, under an unprivileged `rsp` account, listening only on `127.0.0.1:8090`;
3. generates the **owner key** once and stores it in `/etc/rsp-owner-key`;
4. writes the Caddy configuration for the domain and `www`, and gets the certificate;
5. closes the firewall to **22, 80 and 443** only;
6. tests the site and prints the owner key.

Expect it to take a few minutes; most of that is the upload (roughly 150 MB compressed). The last thing it prints is a panel containing the owner key. **Keep it.**

> **IF THE FINAL CHECK IS NOT HTTP 200**
>
> Almost always DNS: Caddy has to be reachable on port 80 at the domain name to be issued a certificate. Re-check step 2, wait, and watch the certificate being issued with `ssh root@<THE-IP> "journalctl -u caddy -f"`. The platform itself is already running; only the certificate is missing.

> **IF THE SERVER IS ALREADY SHOWING A HOLDING PAGE**
>
> That is expected — the domain went up as a static “launching soon” page first. This command replaces what the web server serves, in place: same server, same certificate, no DNS change and no gap where the domain is dead. Nothing needs to be removed first.

> **RUNNING IT AGAIN IS SAFE**
>
> The same command later updates the server to a newer build. The database, the uploaded logos, the encryption key ring and the owner key are all left untouched.

4 Create the first tenant `ONE COMMAND, OVER SSH`

A *tenant* is an IT company using the platform. Anywhere IT is the first one. Public signups are deliberately switched off until the product goes on sale, so the first tenant is created from the command line:

```
ssh root@<THE-IP> "cd /opt/rsp && sudo -u rsp ./RemoteSupport.Platform create-tenant
'Anywhere IT' anywhereit 5 && sudo -u rsp ./RemoteSupport.Platform approve anywhereit"
```

Then add Clint as its portal administrator. Replace `YOUR-PASSWORD` with a strong password:

```
ssh root@<THE-IP> "cd /opt/rsp && sudo -u rsp ./RemoteSupport.Platform add-tech anywhereit
clint@anywhereit.co.za 'YOUR-PASSWORD' 'Clint' --admin"
```

> **THAT PASSWORD IS VISIBLE**
>
> It appears in your shell history and briefly in the server's process list, so treat it as temporary. Tell Clint to change it in the portal at **Technicians** once he has signed in, and clear it from your history.

5 Set up the mailbox the product sends from `ON THE SERVER`

Trial-expiry warnings and portal mail go out from one mailbox. An xneelo mailbox on the domain is fine; create it in konsoleH first if it does not exist, and make sure the domain's MX records point wherever its mail is hosted.

Configure it without the password ever appearing in a command line or in history — `read -s` prompts for it silently:

```
ssh root@<THE-IP>
cd /opt/rsp
read -s RSP_SMTP_PASSWORD; export RSP_SMTP_PASSWORD
sudo -u rsp -E ./RemoteSupport.Platform platform-mail set --host smtp.xneelo.co.za --port
587 --ssl \
  --user noreply@theremotesupportprogram.com \
  --from noreply@theremotesupportprogram.com \
  --from-name "The Remote Support Program" \
  --copy-to clint@anywhereit.co.za
sudo -u rsp ./RemoteSupport.Platform platform-mail test clint@anywhereit.co.za
```

The last line must print `Sent to clint@anywhereit.co.za.` — confirm with Clint that it arrived. The same settings can be entered in the owner portal at `/owner/mail` if you would rather use a browser.

6 Check it, end to end FROM ANY PC

- [ ] `https://theremotesupportprogram.com` loads with a valid padlock, and so does the `www.` version.
- [ ] `http://theremotesupportprogram.com:8090` does **not** answer. It should hang and time out — that is the firewall doing its job.
- [ ] The owner portal at `/owner` accepts the key from step 3.
- [ ] The portal at `/login` accepts `clint@anywhereit.co.za` and the password from step 4.
- [ ] `/get/anywhereit` shows the branded download page and the client package downloads.
- [ ] That client, installed on a machine **outside the office network**, connects and a session runs. This is the real test: it proves the relay works over the public internet, not just the LAN.

Everything downloaded from the portal from now on already carries the tenant key and `wss://theremotesupportprogram.com`, so there is nothing to configure by hand on a customer's machine.

Hand back to Clint

- The **owner key** printed in step 3. Send it over something private — not plain email. It is also readable on the server at `/etc/rsp-owner-key`.
- The **server's IP address** and how to log in.
- The **temporary portal password** from step 4, with a reminder to change it.
- Confirmation that the **test email arrived**.
- Anything in the checklist above that did *not* pass.

If something goes wrong

SYMPTOM	WHAT IT USUALLY IS
The site shows a certificate warning, or does not load at all	DNS is not pointing here yet, or Caddy could not be reached on port 80 to complete the certificate check. Re-check step 2, then `journalctl -u caddy -n 50 --no-pager`.
Caddy answers but shows a 502	The platform is not running. `systemctl status remotesupport-platform` and `journalctl -u remotesupport-platform -n 100 --no-pager`.
The owner portal says the key is not configured	The service did not pick up its settings. `systemctl restart remotesupport-platform`, then check the key in `/etc/rsp-owner-key`.
The deploy command cannot reach the server	SSH. Confirm `ssh root@<THE-IP>` works on its own first; the script needs key-based login, not a password prompt.
Health-check reports arrive as HTML, not PDF	Chrome did not install. Harmless — the reports still send. Fix with `apt install google-chrome-stable` and restart the service.

Useful commands

```
ssh root@<THE-IP> "journalctl -u remotesupport-platform -n 100 --no-pager"
ssh root@<THE-IP> "journalctl -u caddy -n 50 --no-pager"
ssh root@<THE-IP> "systemctl restart remotesupport-platform"
```

Two things never to do on this server

DO NOT DELETE OR MOVE /OPT/RSP/DATA

It holds the database, the uploaded logos and `data/keys`, the encryption key ring. If `data/keys` is lost, **every stored customer password becomes unreadable, permanently** — there is no recovery. Back that folder up off the server before anyone starts relying on it.

DO NOT RUN THE PLATFORM FROM A DIFFERENT FOLDER

It must stay at `/opt/rsp`, installed by the script. Moving it, or running it as a different user, changes how stored passwords are encrypted and they stop decrypting with a misleading "the payload was invalid" error, while everything looks present and correct.

Known limitations — tell the customer, do not try to fix

- **Downloads are not code-signed yet.** Windows SmartScreen warns on the client installer. The certificate is on order; this is expected, not a fault of the install.
- **The portal has no two-factor authentication** and no self-service password reset. A forgotten password is fixed with `set-password` on the server.

- **Nothing monitors the service.** If it stops, no one is told automatically.
